```
> # Appendix B.3: complete self-contained exact rational
  certificate.
  # Includes 5200 central interval checks and 36 auxiliary
  comparisons.
  # Statistic 1 = crank; statistic 2 = rank.
  # No previously defined variables or other programs are
  required.

  AppendixB3 := module()
      option package;

      export VerifyB3;

      local G, Alpha, Beta, Ctab, Dtab, Rpow,
            Down, Up, AddI, MulI, NegI, SubI, ScaleI, DivI,
            PowI, AbsMaxI, ExpAt, Polynomials, HornerI,
            EncloseCell, SectorS, SectorV, CentralError,
  CheckCentral,
            VerifyCentral, Require, ExpLower, CountThird,
            CheckOtherEndpoints;

      G := 10^30;
      Alpha := [41/25, 33/20];
      Beta := 13/10000;

      Ctab := [2, 3, 8, 26, 106, 533, 3134];
      Dtab := [1/2, 5/12, 1/2, 1, 2, 6, 17];
      Rpow := [1, 3/2, 2, 3, 4];

      # Outward rounding to multiples of 10^(-30).

      Down := proc(x::rational)
          return floor(G*x)/G;
      end proc;

      Up := proc(x::rational)
          return -floor(-G*x)/G;
      end proc;

      # An interval is represented by [lower endpoint, upper
  endpoint].

      AddI := proc(A::list, B::list)
          return [Down(A[1]+B[1]), Up(A[2]+B[2])];
      end proc;
```

```
    MulI := proc(A::list, B::list)
        local products;

        products := [
            A[1]*B[1], A[1]*B[2],
            A[2]*B[1], A[2]*B[2]
        ];

        return [
            Down(min(op(products))),
            Up(max(op(products)))
        ];
    end proc;

    NegI := proc(A::list)
        return [-A[2], -A[1]];
    end proc;

    SubI := proc(A::list, B::list)
        return AddI(A, NegI(B));
    end proc;

    ScaleI := proc(c::rational, A::list)
        return MulI([c, c], A);
    end proc;

    DivI := proc(A::list, B::list)
        if B[1] <= 0 and B[2] >= 0 then
            error "division by an interval containing zero: %1",
B;
        end if;

        return MulI(
            A,
            [Down(1/B[2]), Up(1/B[1])]
        );
    end proc;

    PowI := proc(A::list, exponent::nonnegint)
        local result, k;

        result := [1, 1];

        for k from 1 to exponent do
            result := MulI(result, A);
        end do;
```

```
    return result;
end proc;

AbsMaxI := proc(A::list)
    return max(abs(A[1]), abs(A[2]));
end proc;

# Exponential enclosure from Taylor degree 24.

ExpAt := proc(x::rational)
    local term, total, r, tailUpper;

    if x < 0 or x > 13/5 or x/26 >= 1 then
        error "invalid exponential argument: %1", x;
    end if;

    term := 1;
    total := 1;

    for r from 1 to 24 do
        term := term*x/r;
        total := total+term;
    end do;

    tailUpper := (term*x/25)/(1-x/26);

    return [Down(total), Up(total+tailUpper)];
end proc;

# P_j(T) = v^(j)(w), where T = tanh(w/2).
# Coefficients are stored in ascending powers of T.

Polynomials := proc()
    local z, p, result, j, k;

    result := Array(0..5, datatype=anything);
    p := (1-z^2)/4;

    for j from 0 to 5 do
        result[j] := [
            seq(coeff(p, z, k), k=0..degree(p, z))
        ];

        if j < 5 then
            p := expand((1-z^2)*diff(p, z)/2);
```

```
            end if;
        end do;

        return result;
    end proc;

    HornerI := proc(coefficients::list, X::list)
        local result, k;

        result := [0, 0];

        for k from nops(coefficients) by -1 to 1 do
            result := AddI(
                MulI(result, X),
                [coefficients[k], coefficients[k]]
            );
        end do;

        return result;
    end proc;

    # Construct fresh enclosures on one complete closed cell.
    # Return [W_a, enclosure of T, enclosures of
v^(0),...,v^(5)].

    EncloseCell := proc(cellIndex::nonnegint, polys)
        local W, Eleft, Eright, Tint, V, j;

        if cellIndex > 2599 then
            error "invalid cell index: %1", cellIndex;
        end if;

        W := [cellIndex/1000, (cellIndex+1)/1000];

        Eleft := ExpAt(W[1]);
        Eright := ExpAt(W[2]);

        if Eleft[1] < 1 or Eleft[1] > Eleft[2]
           or Eright[1] < 1 or Eright[1] > Eright[2] then
            error "invalid exponential enclosure at cell %1",
                  cellIndex;
        end if;

        Tint := [
            Down((Eleft[1]-1)/(Eleft[1]+1)),
            Up((Eright[2]-1)/(Eright[2]+1))
```

```
        ];

        if Tint[1] < 0 or Tint[1] > Tint[2]
           or Tint[2] >= 1 then
            error "invalid tanh enclosure at cell %1",
cellIndex;
        end if;

        V := [seq(HornerI(polys[j], Tint), j=0..5)];

        for j from 1 to 6 do
            if not type(V[j][1], rational)
               or not type(V[j][2], rational)
               or V[j][1] > V[j][2] then
                error "invalid derivative enclosure: cell %1,
order %2",
                      cellIndex, j-1;
            end if;
        end do;

        if V[1][1] <= 0 then
            error "nonpositive lower bound for v at cell %1",
                  cellIndex;
        end if;

        return [W, Tint, V];
    end proc;

    # Weighted sector bounds.

    SectorS := proc(d::posint, lowerW::rational)
        local q;

        if d > 7 or lowerW < 0 then
            error "invalid weighted-sector arguments";
        end if;

        q := 119/100+27*lowerW/50;

        return min(
            Ctab[d],
            2*factorial(d)*
            (q^(-d-1)+5*q^(-d)/(12*d))
        );
    end proc;
```

```
SectorV := proc(j::nonnegint, lowerW::rational)
    if j > 6 or lowerW < 0 then
        error "invalid derivative-sector arguments";
    end if;

    return min(Dtab[j+1], SectorS(j+1, lowerW));
end proc;

# Upper bound for the central error coefficient.

CentralError := proc(
    j::nonnegint, W::list, absV::list, statistic::posint
)
    local kappa, lam, shiftW, phi2, phiAbs, heat,
          phi, r, ell, coefficient, derivative,
          transfer, g1, g2;

    if j > 2 or (statistic <> 1 and statistic <> 2) then
        error "invalid central-error indices";
    end if;

    kappa := j+1;
    lam := (2*statistic-1)/2;

    if statistic = 1 then
        shiftW := max(0, W[1]-Beta);
        phi2 := [1/3, 2/3, 1/2];
        phiAbs := 1;
    else
        shiftW := W[1];
        phi2 := [7/3, 8/3, 5/2];
        phiAbs := 3/2;
    end if;

    heat :=
        phi2[j+1]/2*SectorV(j+2, shiftW)
        + lam*phiAbs*(
            SectorV(j+3, shiftW)
            + SectorV(j+2, shiftW)
          )
        + lam^2/2*(
            SectorS(kappa+4, shiftW)
            + 2*SectorS(kappa+3, shiftW)
            + SectorS(kappa+2, shiftW)
          );
```

```
        phi := Array(1..4, fill=0, datatype=anything);

        for r from 1 to 4 do
            for ell from 0 to min(r, kappa) do
                coefficient :=
                    binomial(r, ell)*factorial(kappa)
                    /factorial(kappa-ell);

                if r < 4 then
                    derivative := absV[j+r-ell+1];
                else
                    derivative :=
                        Rpow[kappa-ell+1]
                        *SectorV(j+r-ell, W[1]);
                end if;

                phi[r] := phi[r]
                    + coefficient*W[2]^(r-ell)*derivative;
            end do;
        end do;

        transfer := phi[1]+phi[2]+phi[3]+phi[4]/8;

        g1 := (kappa+1)*absV[j+3]+W[2]*absV[j+4];

        g2 := 0;

        for ell from 0 to 2 do
            coefficient :=
                binomial(2, ell)*factorial(kappa+1)
                /factorial(kappa+1-ell);

            g2 := g2
                + coefficient*Rpow[kappa+2-ell]
                  *W[2]^(2-ell)
                  *SectorV(j+4-ell, W[1]);
        end do;

        # The +3 includes minor arc and nonunit major-arc mass.

        return Up(
            transfer+lam*(3*g1+g2)
            +(2+10*Beta)*heat+3
        );
    end proc;
```

```
    # Lower bound on the entire cell and the entire Alpha
interval.

    CheckCentral := proc(
        W::list, V::list, statistic::posint
    )
        local b, lam, twoAlpha, eta, A, linear,
              absV, absA, err, U, enclosure,
              linearLower, losses, loss, low, j;

        if statistic <> 1 and statistic <> 2 then
            error "invalid statistic index";
        end if;

        b := Beta;
        lam := (2*statistic-1)/2;
        twoAlpha := ScaleI(2, Alpha);

        eta := AddI(
            [lam, lam],
            DivI(MulI(W, W), ScaleI(4, Alpha))
        );

        A := [
            SubI(
                SubI(
                    DivI(NegI(V[1]), Alpha),
                    DivI(MulI(ScaleI(3, W), V[2]), twoAlpha)
                ),
                MulI(eta, V[3])
            ),

            SubI(
                SubI(
                    DivI(ScaleI(-5, V[2]), twoAlpha),
                    DivI(MulI(ScaleI(2, W), V[3]), Alpha)
                ),
                MulI(eta, V[4])
            ),

            SubI(
                SubI(
                    DivI(ScaleI(-9, V[3]), twoAlpha),
                    DivI(MulI(ScaleI(5, W), V[4]), twoAlpha)
                ),
                MulI(eta, V[5])
```

```
    )
];

linear := SubI(
    SubI(
        MulI(ScaleI(2, V[2]), A[2]),
        MulI(V[1], A[3])
    ),
    MulI(V[3], A[1])
);

absV := [seq(AbsMaxI(V[j]), j=1..6)];
absA := [seq(AbsMaxI(A[j]), j=1..3)];

err := [
    seq(CentralError(j, W, absV, statistic), j=0..2)
];

U := [seq(Up(absV[j]+b*absA[j]), j=1..3)];

enclosure := ScaleI(
    2, PowI([V[1][1], V[1][1]], 3)
);

linearLower := b*min(linear[1], 0);

enclosure := AddI(
    enclosure, [linearLower, linearLower]
);

losses := [
    b^2*absA[1]*absA[3],

    b^2*(
        2*U[2]*err[2]
        + U[1]*err[3]
        + U[3]*err[1]
    ),

    b^4*(err[2]^2+err[1]*err[3]),

    b^2/4*(U[3]+b^2*err[3])^2
];

for loss in losses do
    enclosure := SubI(enclosure, [loss, loss]);
```

```
        end do;

        low := enclosure[1];

        if low <= 3/10000 then
            error "central bound failed: statistic %1, interval
%2, bound %3",
                  statistic, W, low;
        end if;

        return low;
    end proc;

    VerifyCentral := proc()
        local bounds, polys, minima, locations, counts,
              labels, expectedDigits, expectedMargins,
              cellIndex, cell, W, V, statistic,
              low, previous, digits;

        bounds := Array(1..2, 0..2599, datatype=anything);
        polys := Polynomials();

        minima := Array(1..2, fill=1, datatype=anything);
        locations := Array(1..2, fill=-1, datatype=anything);
        counts := Array(1..2, fill=0, datatype=anything);

        labels := ["CRANK_PASS", "RANK_PASS"];
        expectedDigits := [465010402, 407367051];
        expectedMargins := [465/10^6, 407/10^6];

        previous := 0;

        for cellIndex from 0 to 2599 do
            cell := EncloseCell(cellIndex, polys);
            W := cell[1];
            V := cell[3];

            if W[1] <> previous
               or W[2]-W[1] <> 1/1000 then
                error "central coverage failure at cell %1",
                      cellIndex;
            end if;

            previous := W[2];

            for statistic from 1 to 2 do
```

```
                low := CheckCentral(W, V, statistic);

                bounds[statistic, cellIndex] := low;

                if low < minima[statistic] then
                    minima[statistic] := low;
                    locations[statistic] := cellIndex;
                end if;

                counts[statistic] := counts[statistic]+1;
            end do;
        end do;

        if previous <> 13/5 or counts[1]+counts[2] <> 5200 then
            error "incomplete central domain coverage";
        end if;

        for statistic from 1 to 2 do
            if counts[statistic] <> 2600
               or minima[statistic] <=
expectedMargins[statistic] then
                error "incomplete coverage or insufficient
central margin";
            end if;

            digits := floor(10^12*minima[statistic]);

            if digits <> expectedDigits[statistic]
               or locations[statistic] <> 2599 then
                error "recorded central minimum does not match";
            end if;

            print([
                labels[statistic],
                counts[statistic],
                digits,
                locations[statistic]
            ]);
        end do;

        print([
            "CENTRAL_BOUND_GT_3_OVER_10000_PASS",
            counts[1]+counts[2]
        ]);

        return bounds;
```

```
    end proc;

    # Only an explicitly true comparison is accepted.

    Require := proc(condition, checkName::string)
        local verdict;

        verdict := evalb(condition);

        if not evalb(verdict = true) then
            error "auxiliary certificate failed: %1", checkName;
        end if;

        return 1;
    end proc;

    # Finite Taylor lower bound for exp(x), x >= 0.

    ExpLower := proc(x::rational, maxOrder::nonnegint)
        local term, total, j;

        if x < 0 then
            error "negative argument in ExpLower";
        end if;

        term := 1;
        total := 1;

        for j from 1 to maxOrder do
            term := term*x/j;
            total := total+term;
        end do;

        return total;
    end proc;

    # Closed formula for sum(floor(j/3), j=0..N).
    # This checks the comparison count, not the coefficients
themselves.

    CountThird := proc(N::nonnegint)
        local quotient, remainder;

        quotient := iquo(N, 3);
        remainder := irem(N, 3);
```

```
    return iquo(3*quotient*(quotient-1), 2)
           +(remainder+1)*quotient;
end proc;

CheckOtherEndpoints := proc()
    local b, q, a, d, S1, S2, S3, budget, x,
          checks, entry, count;

    b := Beta;
    q := 2/25;
    a := 34/25;
    d := 9/10;

    S1 := q*(3-2*q)/(1-q)^2;
    S2 := q*(9-11*q+4*q^2)/(1-q)^3;
    S3 := q*(27-44*q+31*q^2-8*q^3)/(1-q)^4;

    budget := (a^2+1/150)*S3/2-a*d*S2+d^2*S1/2;

    x := (127/100)*(2-5/1000);

    checks := [
        [(3/4)^20/factorial(20) > 1/10^21,
         "MOMENT_FACTORIAL_20"],

        [(3/4)^16/factorial(16) > 1/10^16,
         "MOMENT_FACTORIAL_16"],

        [5*10^42*b^29 < 1,
         "SIGNED_POWER_29"],

        [(30*10^21)^2*b^17 < 1,
         "SIGNED_POWER_17"],

        [(2*10^21)^2*b^21 < 1,
         "SIGNED_POWER_21"],

        [35/36+(2/3)*b^9+2*b^8 < 1,
         "MOMENT_CONSTANT_1"],

        [89/60+(1/3)*b^9+2*b^8 < 2,
         "MOMENT_CONSTANT_2"],

        [251/45+2*b^8 < 6,
         "MOMENT_CONSTANT_6"],
```

```
[8/3+b^8 < 3,
 "MOMENT_CONSTANT_3"],

[2/3+b < 7/10,
 "MASS_CONSTANT"],

[6*10^19*b^9 < 1,
 "MINOR_ARC_ERROR"],

[(33/20)/(10^6-1/24) < b^2,
 "BETA_ENDPOINT"],

[4*(33/20)*10^6/(10^6-1/24) < 169/25,
 "CENTRAL_TAIL_OVERLAP"],

[(157/50)^2/6 > 41/25,
 "ALPHA_LOWER_COMPARISON"],

[(22/7)^2/6 < 33/20,
 "ALPHA_UPPER_COMPARISON"],

[4*(41/25) > (64/25)^2,
 "SADDLE_SCALE_LOWER"],

[4*(33/20) < (18/7)^2,
 "SADDLE_SCALE_UPPER"],

[1-99/140 > (27/50)^2,
 "SECTOR_DECAY_CONSTANT"],

[budget = 3228244387/2098807500,
 "TAIL_BUDGET_EXACT"],

[budget < 31/20,
 "TAIL_BUDGET_UPPER"],

[32/25-32/(25*330000)-4/570 > 127/100,
 "CRANK_LOG_SLOPE"],

[(127/100)^2-5/570 > 8/5,
 "TAIL_CURVATURE_LOWER"],

[32/25-32/(25*330000)-5/(3*570) > 127/100,
 "RANK_LOG_SLOPE"],

[a^2 > 90/49,
```

```
     "TAIL_RATIO_UPPER_CONSTANT"],

    [256/283-3/(2*570) > d,
     "TAIL_SLOPE_LOWER_CONSTANT"],

    [15/(4*570) < 1/150,
     "TAIL_CURVATURE_CORRECTION"],

    [((283/200)^2-2)*330000 > 10,
     "TAIL_SCALE_GAP"],

    [6001 < 10^6/150-1/10,
     "TAIL_PARAMETER_DOMAIN"],

    [(9592/10000)^2 > 23/25,
     "EXPONENTIAL_WEIGHT_RATIO"],

    [(1-(9999/10000)^2)*330000 > 4,
     "EXPONENTIAL_QUADRATIC_GAP"],

    [(64/25)*(9999/10000-9592/10000) > 1/10,
     "EXPONENTIAL_LINEAR_GAP"],

    [ExpLower(x, 10) > 25/2,
     "EXPONENTIAL_BOUND_DEGREE_10"],

    [ExpLower(57, 50) > 10^24,
     "EXPONENTIAL_BOUND_DEGREE_50"],

    [25000*330000^3/10^24 < 1/1000,
     "TAIL_ERROR_BUDGET"],

    [8/5-31/20-1/1000 > 0,
     "TAIL_POSITIVE_MARGIN"],

    [CountThird(10^6-1)+CountThird(10^6-8)-2316
        = 333329997696,
     "FINITE_COMPARISON_COUNT"]
];

if nops(checks) <> 36 then
    error "incorrect number of auxiliary comparisons";
end if;

count := 0;
```

```
        for entry in checks do
            count := count+Require(entry[1], entry[2]);
        end do;

        if count <> 36 then
            error "incomplete auxiliary certificate";
        end if;

        print([
            "MOMENT_DOMAIN_AND_TAIL_ENDPOINTS_PASS",
            count
        ]);

        return count;
    end proc;

    VerifyB3 := proc()
        local endpointCount, centralBounds;

        endpointCount := CheckOtherEndpoints();
        centralBounds := VerifyCentral();

        print("APPENDIX_B3_PASS");

        return [endpointCount, centralBounds];
    end proc;

end module:

# Run the complete independent B.3 certificate.
AppendixB3Result := AppendixB3:-VerifyB3():
```

```
["MOMENT_DOMAIN_AND_TAIL_ENDPOINTS_PASS", 36]
["CRANK_PASS", 2600, 465010402, 2599]
["RANK_PASS", 2600, 407367051, 2599]
["CENTRAL_BOUND_GT_3_OVER_10000_PASS", 5200]
"APPENDIX_B3_PASS"
```

```
>
```